

Approximate Boyer-Moore String Matching for Small Alphabets

Leena Salmela · Jorma Tarhio · Petri Kalsi

Received: 8 September 2008 / Accepted: 28 January 2009 / Published online: 18 February 2009
© Springer Science+Business Media, LLC 2009

Abstract Recently a new variation of approximate Boyer-Moore string matching was presented for the k -mismatch problem. The variation, called FFAST, is specifically tuned for small alphabets. We further improve this algorithm gaining speedups in both preprocessing and searching. We also present three variations of the algorithm for the k -difference problem. We show that the searching time of the algorithms is average-optimal and the preprocessing also has a lower time complexity than FFAST. Our experiments show that our algorithm for the k -mismatch problem is about 30% faster than FFAST and the new algorithms compare well against other state-of-the-art algorithms for approximate string matching.

Keywords Approximate string matching · Edit distance · Hamming distance · Complexity of algorithms · Biological sequences

1 Introduction

We consider two variations of approximate string matching, the k -mismatch problem and the k -difference problem. In both of the problems, we have a pattern $p = p_0, \dots, p_{m-1}$ of m characters drawn from an alphabet Σ of size σ and a text $t = t_0, \dots, t_{n-1}$ of n characters over the same alphabet. We need to find all such substrings of the text that the distance between the substring and the pattern is at most k .

Work supported by Academy of Finland.

An earlier version of this paper appeared in *Proceedings of String Processing and Information Retrieval* (Oct. 29–31, 2007) pp. 173–183.

L. Salmela (✉) · J. Tarhio · P. Kalsi

Department of Computer Science and Engineering, Helsinki University of Technology,
P.O. Box 5400, 02015 TKK, Espoo, Finland
e-mail: lsalmela@cs.hut.fi

J. Tarhio

e-mail: tarhio@cs.hut.fi

In the k -difference problem the distance between two strings is the standard edit distance where mismatches, deletions, and insertions are allowed. The k -mismatch problem is a more restricted one using the Hamming distance where only mismatches are allowed.

Several algorithms [13] for both variations of approximate string matching have been presented. Many of the algorithms have been developed with text data in mind and these algorithms do not necessarily work well with a small alphabet. Recently developing algorithms for small alphabets has attracted attention as approximate searching of large volumes of gene sequences, which are long strings over the DNA alphabet, $\{a, c, g, t\}$, has become common. Such a search routine is implemented for example by the *primersearch* program of the European Molecular Biology Open Software Suit (EMBOSS) [16]. The program searches for a pair of primers, which are short sequences of DNA, in a collection of gene sequences and extracts all those sequences that contain the primers with at most k mismatches or differences.

The approximate Boyer-Moore (ABM) algorithm [17] is an adaptation of the Boyer-Moore-Horspool algorithm [9] to approximate matching. ABM performs well on moderately large alphabets and low error levels. ABM was originally not designed for small alphabets and in fact it performs rather poorly on them. Liu et al. [10] tuned the k -mismatch version of ABM for smaller alphabets. Their algorithm, called FFAST, has a stronger shift function which makes it faster than ABM.

In this paper we introduce improvements to the FFAST algorithm gaining considerable speedups in both preprocessing and search times and show that our version of the algorithm is easily modified to handle the k -difference problem. We analyze the complexity of the algorithms and show that the searching time of our algorithms and FFAST is average-optimal and that our simpler preprocessing has a lower time complexity than that of FFAST.

2 Preliminaries

2.1 Dynamic Programming

Dynamic programming is a well known technique to calculate the edit distance between two strings, $a = a_0, \dots, a_{m-1}$ and $b = b_0, \dots, b_{n-1}$. The dynamic programming table D of size $(m + 1) \times (n + 1)$ is initialized by setting $D[i, 0] = i$ for $0 \leq i \leq m$ and $D[0, j] = j$ for $0 \leq j \leq n$. The rest of the entries are filled with the recurrence relation:

$$D[i, j] = \min \begin{cases} D[i - 1, j - 1] + \alpha, \\ D[i - 1, j] + 1, \\ D[i, j - 1] + 1 \end{cases} \quad \text{where } \alpha = \begin{cases} 0 & \text{if } a_{i-1} = b_{j-1}, \\ 1 & \text{otherwise.} \end{cases}$$

Here the first alternative $D[i - 1, j - 1] + \alpha$ takes care of both mismatches and matches, and the second and third alternatives represent insertions and deletions. From this table we can get the edit distances between the prefixes of a and the prefixes of b . In other words, the starting positions of the compared substrings are fixed to the beginning of the strings. The entry $D[m, n]$ gives the minimum edit distance

Fig. 1 The edit distance table D for the strings $a = \text{"cata"}$ and $b = \text{"tgga"}$ with different initializations

		D									D						
		t g g c a a									t g g c a a						
$i \backslash j$		0	1	2	3	4	5	6	$i \backslash j$		0	1	2	3	4	5	6
0	0	0	1	2	3	4	5	6	0	0	0	0	0	0	0	0	0
c	1	1	1	2	3	3	4	5	c	1	0	1	1	1	0	1	1
a	2	2	2	2	3	4	3	4	a	2	0	1	2	2	1	0	1
t	3	3	2	3	3	4	4	4	t	3	0	0	1	2	2	1	1
a	4	4	3	3	4	4	4	4	a	4	0	1	1	2	3	2	1

between the two strings. The entry $D[i, n]$ where $0 \leq i \leq m$, gives the minimum edit distance between the prefix $a_0, \dots, a_{i-1}$ and the string b . Similarly, $D[m, j]$ where $0 \leq j \leq n$, gives the minimum edit distance between the string a and the prefix $b_0, \dots, b_{j-1}$. Figure 1 gives an example of edit distance calculation for the strings “cata” and “tgga”.

We can also initialize the table differently by setting $D[i, 0] = 0$ for $0 \leq i \leq m$ and $D[0, j] = 0$ for $0 \leq j \leq n$ and then fill the table with the same recurrence relation as before. Now the starting positions of the compared substrings can vary. If $m \leq n$, $D[m, n]$ gives the minimum edit distance when aligning the string a against any suffix of b . Furthermore, the entry $D[m, j]$ gives the minimal edit distance when aligning the string a against any suffix of the prefix $b_0, \dots, b_{j-1}$ where deletions in the beginning of a are free. Figure 1 also shows an example of the edit distance table with this initialization.

Note that the original approximate string matching algorithm based on dynamic programming initializes the first column by setting $D[i, 0] = i$ for $0 \leq i \leq m$ and the first row by setting $D[0, j] = 0$ for $0 \leq j \leq n$. In this original setting, the first string must be fully contained inside the second string while we will consider approximate matching in sliding windows where the strings may overlap each other in different ways.

Dynamic programming can also be used to calculate Hamming distances. Because the Hamming distance only allows mismatches, it is only possible to calculate the Hamming distance of two strings of equal lengths. Thus only initializing the first row and column to 0 makes sense. Also the recurrence relation for filling the table is different:

$$D[i, j] = D[i - 1, j - 1] + \alpha \quad \text{where } \alpha = \begin{cases} 0 & \text{if } a_{i-1} = b_{j-1}, \\ 1 & \text{otherwise.} \end{cases}$$

Suppose that $m \leq n$. Then the entry $D[m, n]$ gives the Hamming distance of aligning the string a against the end of b . The entries $D[m, j]$, where $0 \leq j \leq n$, give the Hamming distance for aligning the string a against the end of a prefix $b_0, \dots, b_{j-1}$. Figure 2 shows an example of the Hamming distance table for strings “cata” and “tgga”.

2.2 Previous Algorithms

So far many algorithms have been developed based on Boyer-Moore string matching [5] for the k -mismatch problem. Here we consider mainly ABM [17] and FAFAST

Fig. 2 The Hamming distance table D for strings $a = \text{"cata"}$ and $b = \text{"tgcaaa"}$

		D						
		t g g c a a						
$i \backslash j$		0	1	2	3	4	5	6
	0	0	0	0	0	0	0	0
c	1	0	1	1	1	0	1	1
a	2	0	1	2	2	2	0	1
t	3	0	0	2	3	3	3	1
a	4	0	1	1	3	4	3	3

[10], but two other variations developed by Baeza-Yates and Gonnet [3] and El-Mabrouk and Crochemore [7] are worth mentioning. The shift function of the Baeza-Yates-Gonnet algorithm is based on the triangular inequality and it achieves $\mathcal{O}(nk)$ average case complexity for searching and $\mathcal{O}(m(m-k))$ time for preprocessing. The El-Mabrouk-Crochemore algorithm applies the Shift-Add approach [2]. Its average case complexity for searching is $\mathcal{O}(n + kn/(m-k))$ and its complexity for preprocessing is $\mathcal{O}(m + \sigma)$. The average case complexity of searching in ABM is $\mathcal{O}(nk(1/(m-k) + k/\sigma))$ and the preprocessing cost is $\mathcal{O}(m + k\sigma)$. The preprocessing cost of FFAST is $\mathcal{O}(q((m-k)\sigma^q + m))$ and the analysis in this work establishes the average case complexity of searching in FFAST to be $\mathcal{O}(n(\log_\sigma m + k)/m)$ which has been shown to be optimal for approximate string matching [6].

Typically algorithms of Boyer-Moore type have two phases: preprocessing of the pattern and searching of its occurrences in the text. ABM uses the bad character rule for shifting and is thus a direct generalization of the Boyer-Moore-Horspool algorithm [9]. Instead of stopping at the first mismatch in the matching loop, the algorithm stops at the $k+1$ st mismatch or when an occurrence of the whole pattern is found. The shift is calculated considering $k+1$ characters currently aligned with the end of the pattern. The shift is the minimum of the precomputed shifts for those $k+1$ characters. After shifting, at least one of these characters will be aligned correctly with the pattern or the pattern has been shifted past the first of these characters.

FFAST is an improved version of ABM for small alphabets using a variation of the Four-Russians technique [1, 11, 18] to speed up the search. Instead of minimizing $k+1$ shifts during search, it uses a precomputed shift table for a q -gram aligned with the end of the pattern, where $q > k$ is a parameter of the algorithm. (Liu et al. [10] use the notation $(k+x)$ -gram where $x \geq 1$.) The shift table is calculated so that after the shift at least $q-k$ characters are aligned correctly or the pattern is shifted past the last q -gram of the alignment. It is obvious that this stronger requirement leads to longer shifts in most situations, when $q-k > 1$ holds, and the shift is never shorter than the shift of ABM. Note that for $q = k+1$ the length of the shift is the same for both of the algorithms, but ABM performs the minimization of $k+1$ shifts during searching while this minimization is done during preprocessing in FFAST. So the algorithms are different even for $q = k+1$. The optimal value of q for maximum searching speed depends on other problem parameters and the computing platform. However, an increment of q makes the preprocessing time grow. FFAST presents a clear improvement on solving the k -mismatch problem on small alphabets as compared to the ABM algorithm. The preprocessing phase of FFAST is advanced because it includes the minimization step of ABM.

A version of the ABM algorithm can also solve the k -difference problem while the other Boyer-Moore type algorithms discussed above are limited to the k -mismatch problem. For the k -difference problem the preprocessing cost of ABM is $\mathcal{O}((k + \sigma)m)$ and the average case complexity for searching in ABM is

$$\mathcal{O}\left(\frac{\sigma}{\sigma - 2k}kn\left(\frac{k}{\sigma + 2k^2} + \frac{1}{m}\right)\right)$$

if $2k + 1 < \sigma$.

Other approximate string matching algorithms utilizing the backward matching paradigm are approximate BNDM (ABNDM) [14] and the backward matching versions of the algorithm by Fredriksson and Navarro [8]. Both of these algorithms can solve both versions of the approximate string matching problem. ABNDM is a bit-parallel simulation of an automaton that identifies approximate matches of factors of the pattern in each alignment and then shifts the pattern according to the found matches. The k -difference version of ABNDM is a filtering algorithm so the found matches must be verified. The average case complexity of searching in ABNDM is

$$\mathcal{O}\left(n\frac{\alpha + \alpha^* \log_{\sigma} m/m}{(1 - \alpha)\alpha^* - \alpha}\right)$$

where $\alpha = k/m$ is the error level and α^* is the maximum error level for which the probability of a random pattern matching a string with at most k differences is exponentially decreasing with m [13]. The preprocessing time of ABNDM is $\mathcal{O}(\sigma + m)$.

The algorithm by Fredriksson and Navarro [8] reads non-overlapping q -grams (ℓ -grams in the original paper) in an alignment and with the help of preprocessed tables determines the minimum number of mismatches or differences for aligning the q -grams with the pattern in some way. When the minimum number of mismatches or differences exceeds k , the pattern is shifted so that the first of these q -grams is no longer aligned with the pattern. The potential matches must be verified. The algorithm has only been analyzed for the k -difference problem. The average case complexity of searching is $\mathcal{O}(n(k + \log_{\sigma} m)/m)$ if $k/m = 1/2 + \mathcal{O}(1/\sqrt{\sigma})$ and the complexity of preprocessing is $\mathcal{O}(m\sigma^q)$.

3 Algorithm for the k -Mismatch Problem

Our aim is to develop a faster algorithm for small alphabets based on FFAST which uses a q -gram for shifting. We make two major changes to FFAST. First, we implement a simpler and faster preprocessing phase based on dynamic programming. FFAST counts the number of mismatches in the q -gram aligned with the end of the pattern during the searching phase. Our second improvement is to compute this number during preprocessing, which improves the searching speed.

For each q -gram $s_0, \dots, s_{q-1}$, the preprocessing phase computes the Hamming distance when aligning the q -gram against the end of all prefixes of the pattern. As explained in Sect. 2.1, we get all these Hamming distances using dynamic programming by initializing the first row and column of the dynamic programming table to 0,

and filling the rest of the table using the recurrence relation for Hamming distance. The bottom row $D[q, j]$, where $0 \leq j \leq m$, will then give the needed Hamming distances.

As an example on how these precalculated tables are used during the searching phase, let us consider a situation where a pattern $p = \text{"tggcaa"}$ has been aligned with the text string "tgcata" , and $k = 2$ and $q = 4$ holds. The reference q -gram is now "cata" , and the corresponding Hamming distance table of size $(q + 1) \times (m + 1)$, calculated during preprocessing, is shown in Fig. 2. First of all, we see that the last cell $D[q, m] = 3 > k$, and therefore it is not possible to find a match at this position, as already the suffix of the aligned text string contains too many mismatches. Otherwise, we would have to check for a match by examining the amount of mismatches in the beginning of the aligned string.

We will also look at the bottom row of the table, and find the rightmost cell $D[q, j]$ with a value $h \leq k$, except for the last cell $D[q, m]$. This is the next possible candidate for aligning the pattern with the text with less than k mismatches and the correct shift is equal to $m - j$. In our example, the cell $D[q, 2] = 1$, and we would shift the pattern by $6 - 2 = 4$ positions to get the next alignment.

We do not need the whole table to obtain this information, so we just store the calculated Hamming distance for each generated q -gram in a table M which is indexed by a number obtained by transforming the q -gram to an integer. The precalculated shifts are stored in a table S_q . During the searching phase we convert the last q -gram of the aligned text string into an index y to the tables, and check for an occurrence if $M[y] \leq k$. Finally, we shift the pattern according to $S_q[y]$.

We can improve the preprocessing time by applying the technique used previously by Fredriksson and Navarro [8] for approximate matching and Navarro et al. [15] for indexed approximate matching. If the q -grams are generated in the lexicographical order, the dynamic programming table differs only by the last few rows in most cases. Therefore we can speed up the preprocessing if we only recalculate the last rows of the table at each step, starting from the first changed character.

This can be implemented by traversing the tree built of all q -grams in depth first order. Nodes at the i th level of the tree correspond to strings of length i . Thus there are σ^i nodes on level i and the total number of nodes in the tree is

$$\sum_{i=0}^q \sigma^i = \frac{\sigma^{q+1} - 1}{\sigma - 1} = \mathcal{O}(\sigma^q).$$

If we have the dynamic programming table for a node in the tree, the tables for the children nodes can be obtained by calculating one more row to the dynamic programming table taking $\mathcal{O}(m)$ time per child so calculating the dynamic programming tables for all nodes in the tree takes $\mathcal{O}(\sigma^q m)$ time. At the leaf nodes we have the dynamic programming table for the corresponding q -gram and we need to figure out the number of mismatches entered to table M and the shift value entered to table S_q which takes $\mathcal{O}(m)$ time. The extra calculation needed at leaf nodes is thus $\mathcal{O}(m\sigma^q)$ because there are σ^q leaf nodes. Therefore the time complexity of the preprocessing phase is $\mathcal{O}(m\sigma^q)$. Note that if we implement the traversing of the tree by recursion, we do not need to explicitly build the tree. The preprocessing time of FFAST is $\mathcal{O}(q((m - k)\sigma^q + m))$ so asymptotically our preprocessing is faster by a factor of q .

This algorithm for the k -mismatch problem is called Algorithm 1 and the pseudo code of the algorithm is shown in Fig. 3. The shift behaviors of Algorithm 1 and FFAST are exactly the same. In FFAST the number of mismatches in the last q -gram of an alignment is computed during the searching phase whereas in Algorithm 1 this is fetched from a table. However, we still need to read the q -gram and thus the time complexity of the search phase of Algorithm 1 is the same as in FFAST.

preprocess_helper (p, m, k, q, i, s)

```

1.  if ( $i = q + 1$ )
2.     $M[s] \leftarrow D[q, m]$ 
3.    for ( $j = m - 1$  down to 0)
4.      if ( $D[q, j] \leq k$ )
5.         $S_q[s] \leftarrow m - j$ 
6.        break
7.  else
8.    for ( $c \in \Sigma$ )
9.      for ( $j = 1$  to  $m$ )
10.          $D[i, j] \leftarrow D[i - 1, j - 1] + \alpha$  where  $\alpha = \begin{cases} 0 & \text{if } c = p_{j-1}, \\ 1 & \text{otherwise} \end{cases}$ 
11.    preprocess_helper( $p, m, k, q, i + 1, s + c$ )

```

preprocess (p, m, k, q)

```

1.  for ( $i = 0$  to  $q$ )
2.     $D[i, 0] \leftarrow 0$ 
3.  for ( $j = 0$  to  $m$ )
4.     $D[0, j] \leftarrow 0$ 
5.  preprocess_helper( $p, m, k, q, 1, ""$ )

```

search (t, n, k, q)

```

1.   $s \leftarrow m - 1$ 
2.  while ( $s < n$ )
3.    if ( $(M[t_{s-q+1} \dots t_s] \leq k)$  /* possible occurrence */
4.       $c = M[t_{s-q+1} \dots t_s]$ 
5.      for ( $i = 1$  to  $m - q$ )
6.        if ( $t_{s-q-i+1} \neq p_{m-q-i}$ )
7.           $c = c + 1$ 
8.          if ( $c > k$ ) break
9.      if ( $c \leq k$ )
10.        Report an occurrence at  $t_{s-m+1} \dots t_s$ 
        with  $c$  mismatches
11.     $s \leftarrow s + S_q[t_{s-q+1} \dots t_s]$ 

```

Fig. 3 Algorithm 1 preprocessing and search phases

Implementation note For maximum performance it is crucial how the value of a q -gram is computed during searching. We mapped the ASCII values of DNA characters to integers $\{0, 1, 2, 3\}$ and used the following loop to construct a bit representation of a q -gram:

```
bits = 0
for (i = 0 to q - 1)
    bits = (bits << 2) | map( $s_i$ )
```

where $s = s_0, \dots, s_{q-1}$ is the q -gram, map is a function that maps the DNA characters to integers and bits is the bit representation of the q -gram.

4 Algorithms for the k -Difference Problem

Algorithm 1 can be easily modified to solve the k -difference problem. As in Algorithm 1 we construct a dynamic programming table D' for each q -gram and the pattern during preprocessing. We do not store these tables but only use them to fill the tables M and S_q . To get the values for these tables, we need the edit distance of each q -gram aligned with the end of each prefix of the pattern. As explained in Sect. 2.1 we get these values by initializing the first row and column of the table D' to 0 and applying the recurrence relation for edit distance. The needed edit distances are then found in the last row $D'[q, j]$ where $0 \leq j \leq m$. For each q -gram, we store the minimum number of mismatches, insertions, and deletions needed to align the q -gram against the end of the pattern to the table M . This value is obtained from $D'[q, m]$. To enter the shift values to S_q , we find the largest $j < m$, such that $D'[q, j] \leq k$ and enter $m - j$ to the table S_q .

The searching phase starts by aligning the pattern against the text prefix ending at position $m - k - 1$. When examining an alignment ending at position s , all matches ending before that position have been reported. If $M[t_{s-q+1} \dots t_s] \leq k$, we need to check for an occurrence ending at s by using dynamic programming. We initialize the first row and column of the table to 0, because we do not know the exact position of the start of the occurrence. The maximum length of the occurrence is $m + k$ so it is sufficient to construct a $(m + k + 1) \times (m + 1)$ edit distance table D with the currently aligned text $t_{s-(m+k)+1} \dots t_s$ against the pattern. A match will be reported, if $D[m + k, m] \leq k$. After this operation we will shift the pattern according to S_q . In order to observe correctly an occurrence of the pattern in the beginning of the text, we assume that $t_{-2k}, \dots, t_{-1}$ hold a character not in the pattern.

The preprocessing phase can be improved using the same technique as for Algorithm 1. The only difference is that when filling the dynamic programming table we now use the recurrence relation for edit distance. The modification of Algorithm 1 for the k -difference problem is called Algorithm 2.

Example tables for the k -difference problem are shown in Fig. 4, using a pattern “tgccaa”, a text string “aaggcata” and parameters $k = 2$ and $q = 4$. We can see from the first table that $S_q[\text{“cata”}] = 6 - 5 = 1$ and $M[\text{“cata”}] = D'[q, m] = 1$. Therefore, we would construct a table D , and find that $D[m + k, m] = 2 \leq k$, and report a match at position s . We would continue the search by shifting the pattern by one.

In the k -mismatch problem we did not need to reread the last q characters from the text alignment when checking for an occurrence. Instead, we had stored the number

								D								D_r							
D'																							
t g g c a a								t g g c a a								a a c g g t							
$i \backslash j$	0	1	2	3	4	5	6	$i \backslash j$	0	1	2	3	4	5	6	$i \backslash j$	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	2	3	4	5	6
c 1	0	1	1	1	0	1	1	a 1	0	1	1	1	1	0	0	a 1	1	0	1	2	3	4	5
a 2	0	1	2	2	1	0	1	a 2	0	1	2	2	2	1	0	t 2	2	1	1	2	3	4	4
t 3	0	0	1	2	2	1	1	g 3	0	1	1	2	3	2	1	a 3	3	2	1	2	3	4	5
a 4	0	1	1	2	3	2	1	g 4	0	1	1	1	2	3	2	c 4	4	3	2	1	2	3	4
								c 5	0	1	2	2	1	2	3	g 5	5	4	3	2	1	2	3
								a 6	0	1	2	3	2	1	2	g 6	6	5	4	3	2	1	2
								t 7	0	0	1	2	3	2	2	a 7	7	6	5	4	3	2	2
								a 8	0	1	1	2	3	3	2	a 8	8	7	6	5	4	3	3

Fig. 4 Normal and reversed edit distance tables for k -difference problem ($k = 2, q = 4$) with the pattern “tgggcaa” and the aligned text “aaggcata”. Sizes of the tables are $(q + 1) \times (m + 1)$ for D' and $(m + k + 1) \times (m + 1)$ for D and D_r

of mismatches in the table M and we could extend the match based on that information. For the k -difference problem the situation is not quite as simple because we need to compute the dynamic programming table to check for an occurrence. The problem with Algorithm 2 is that when checking for an occurrence the aligned text is read forward while during the preprocessing phase we have generated the dynamic programming table for the last characters of the pattern. In order to use that information and avoid rereading the last q characters we need to reverse the calculation of the dynamic programming table so that we start building the table from the end of the pattern and the text string.

Suppose that we want to check for an occurrence ending at position s . We can build the edit distance table for the reversed pattern $p_{m-1}, \dots, p_0$ and the reversed text substring $t_s, \dots, t_{s-m-k+1}$. For the reversed strings the starting position of the occurrence is fixed so we initialize the table by setting $D[0, j] = j$ and $D[i, 0] = i$ for $i \in [0, m+k], j \in [0, m]$. This reversed table gives equivalent results when it comes to calculating the actual edit distance between the pattern and the aligned text string. When this reversed edit distance table D_r has been finished, we have to search for a match at the last column. To be exact, we need to check $2k+1$ different cells of the table for a possible match, because the match can contain up to k insert or delete operations, and the match length can therefore vary. All possible matches that end in the character t_s will be found in the last $2k+1$ cells of the last column of the reversed table. We can either report the first match with at most k differences, or search for the match with the minimum differences. The current alignment $t_{s-(m+i)+1} \dots t_s$ matches the pattern $p_0 \dots p_{m-1}$ with at most k differences, if

$$D_r[m + i, m] \leq k$$

for any $i \in -k \dots k$. Figure 4 shows an example of the reversed edit distance table.

To avoid rereading the last q -gram of an alignment for constructing the edit distance table, we can calculate the reversed edit distance table for each q -gram and the pattern during preprocessing. During searching we can then check for a complete

occurrence by filling the rest of the table rows from t_{s-q} down to $t_{s-(m+k-1)}$. We can therefore store the last row of the reversed table $D_r[q, j]$, $j \in [0, m]$ for each q -gram during the preprocessing phase. This row can then be used to fill up the rest of the table by dynamic programming during the search phase, when the aligned text needs to be checked for an occurrence, and we do not need to run the dynamic programming for the whole table every time. However, we still need the normal edit distance table to obtain the values for tables S_q and M .

When verifying an occurrence, we need the last $2k + 1$ rows to be able to check the cells $D_r[m + i, m]$ where $i \in -k \dots k$. We have access only to the last $m + k - q + 1$ rows as the first $q + 1$ rows were computed during preprocessing and only the last of these is saved. Thus choosing $q \leq m - k$ guarantees that all the needed rows will be available when verifying an occurrence.

We modify Algorithm 2 to use the reversed table during the search phase, and we also store the last row of the reversed tables generated during the preprocessing phase. The new algorithm is called Algorithm 3, and its pseudo code is given in Fig. 5. For simplicity, the preprocessing part of the pseudo code does not use the optimization of generating the q -grams in lexicographic order and recalculating the dynamic programming table only for those rows that have changed.

The preprocessing phase of Algorithm 2 has the same time complexity as that of Algorithm 1 as the only difference is that the dynamic programming table is filled using the recurrence relation for edit distance in Algorithm 2 while the preprocessing phase of Algorithm 1 uses the recurrence relation for Hamming distance. In Algorithm 3, we need to calculate both the original dynamic programming table and the reversed one. Because a q -gram is read in opposite directions when calculating these two tables we have to enumerate the q -grams twice. However, the asymptotic time complexity remains the same.

The shifts in the searching phase of Algorithms 2 and 3 are somewhat shorter than in Algorithm 1 because the probability of two strings matching with distance less than k is higher when using the edit distance than when using the Hamming distance.

In the worst case Algorithms 2 and 3 degenerate to calculating the dynamic programming table at each position of the text. Thus the worst case complexity of Algorithms 2 and 3 is $\mathcal{O}(nm^2)$. The worst case complexity can be improved to $\mathcal{O}(nm)$ by doing the verification forward and incrementally. This technique has previously been used to improve the worst case complexity of ABM [17] as well as many other approximate string matching algorithms based on filtering [13]. We now store the end position of the previous verification and also the last row of the dynamic programming table of the previous verification. When we need to perform another verification, we first check if the starting position of this new verification is before the end position of the previous verification. If so, we continue the verification from the end position of the previous verification. This guarantees that we never traverse a text position twice for verification purposes. Now in the worst case the pattern is always shifted by one position and a verification is triggered in each position. To compute the shifts we read $\mathcal{O}(qn)$ characters and the verification cost in each position is $\mathcal{O}(m)$ as we add one new row to the dynamic programming table. Thus the worst case complexity is $\mathcal{O}(nm)$. This modification of the algorithm is called Algorithm 4.

preprocess (p, m, k, q)

```

1. for ( $i=0$  to  $q$ )
2.    $D'[i, 0] \leftarrow 0$ 
3.    $D_r[i, 0] \leftarrow i$ 
4. for ( $j=0$  to  $m$ )
5.    $D'[0, j] \leftarrow 0$ 
6.    $D_r[0, j] \leftarrow j$ 
7. for ( $s=s_0 \dots s_{q-1} \in \Sigma^q$ )
8.   for ( $i=1$  to  $q$ )
9.     for ( $j=1$  to  $m$ )
10.       $D'[i, j] \leftarrow \min \begin{cases} D'[i-1, j-1] + \alpha, \\ D'[i-1, j] + 1, \\ D'[i, j-1] + 1, \end{cases} \quad \alpha = \begin{cases} 0 & \text{if } s_{i-1} = p_{j-1}, \\ 1 & \text{otherwise} \end{cases}$ 
11.       $D_r[i, j] \leftarrow \min \begin{cases} D_r[i-1, j-1] + \alpha, \\ D_r[i-1, j] + 1, \\ D_r[i, j-1] + 1, \end{cases} \quad \alpha = \begin{cases} 0 & \text{if } s_{q-i} = p_{m-j}, \\ 1 & \text{otherwise} \end{cases}$ 
12.     $M[s] \leftarrow D'[q, m]$ 
13.     $\text{lastRow}[s] \leftarrow D_r[q]$ 
14.    for ( $j=m-1$  down to  $0$ )
15.      if ( $D'[q, j] \leq k$ )
16.         $S_q[s] \leftarrow m - j$ 
17.        break

```

search (t, n, k, q)

```

1. for ( $i=0$  to  $m+k$ )
2.    $D_r[i, 0] \leftarrow i$ 
3. for ( $j=0$  to  $m$ )
4.    $D_r[0, j] \leftarrow j$ 
5.  $s \leftarrow m - k - 1$ 
6. while ( $s < n$ )
7.   if ( $M[t_{s-q+1} \dots t_s] \leq k$ ) /* possible occurrence */
8.      $D_r[q] \leftarrow \text{lastRow}[t_{s-q+1} \dots t_s]$ 
9.     for ( $j=1$  to  $m$ )
10.      for ( $i=q+1$  to  $m+k$ )
11.        $D_r[i, j] \leftarrow \min \begin{cases} D_r[i-1, j-1] + \alpha, \\ D_r[i-1, j] + 1, \\ D_r[i, j-1] + 1, \end{cases}$ 
12.       where  $\alpha = \begin{cases} 0 & \text{if } t_{s-i+1} = p_{m-j}, \\ 1 & \text{otherwise} \end{cases}$ 
13.       if ( $D_r[m+i, m] \leq k$  for any  $i \in -k \dots k$ )
14.         Report match at  $t_{s-(m+i)+1} \dots t_s$ 
15.         with  $D_r[m+i, m]$  differences
16.        $s \leftarrow s + S_q[t_{s-q+1} \dots t_s]$ 

```

Fig. 5 Algorithm 3 preprocessing and search phases

To compute a new row to the dynamic programming table, we only need the previous row of the table so it is enough to save the previous and current rows of the table. The space needed for verification in Algorithm 4 is thus $\mathcal{O}(m)$.

5 Analysis

In the worst case Algorithm 1 reads m characters in each alignment and always shifts the pattern by one position. Thus the worst case complexity of Algorithm 1 is $\mathcal{O}(nm)$. The worst case complexity of Algorithms 2 and 3 is $\mathcal{O}(nm^2)$ and the worst case complexity of Algorithm 4 is $\mathcal{O}(nm)$ as stated in the previous section.

Let us then analyse the average case complexity of the algorithms. Here we assume the standard random string model where each character of the text and the pattern is chosen independently and uniformly at random. We will make use of the following Lemmas originally proved by Chang and Marr [6] but here we use the rewritten form by Fredriksson and Navarro [8] which is more convenient for our purposes.

Lemma 1 *The probability that two random q -grams have a common subsequence of length $(1 - c)q$ is at most $a\sigma^{-dq}/q$, for constants $a = (1 + o(1))/(2\pi c(1 - c))$ and $d = 1 - c + 2c \log_{\sigma} c + 2(1 - c) \log_{\sigma} (1 - c)$. The probability decreases exponentially for $d > 0$, which surely holds if $c < 1 - e/\sqrt{\sigma}$.*

Lemma 2 *If s is a q -gram that matches inside a given string p (longer than q) with less than cq differences, then s has a common subsequence of length $q - cq$ with some q -gram of p .*

Theorem 1 *If $k/m < 1/2 - \mathcal{O}(1/\sqrt{\sigma})$, the average case complexity of searching in FFAST and Algorithms 1, 2, 3, and 4 is $\mathcal{O}(n(\log_{\sigma} m + k)/m)$ if we choose $q = \Theta(\log_{\sigma} m + k)$.*

Proof Let us choose a constant c such that $d > 0$ and therefore the probability of matching given by Lemma 1 decreases exponentially when q is increased. Given the number of mismatches or differences $k \leq cq$, the probability that a q -gram matches inside the pattern with at most k mismatches or differences is at most $ma/(q\sigma^{dq})$ because there are less than m q -grams in the pattern and by Lemma 2 one of them has to have a common subsequence of length $q - k \geq q - cq$ with the q -gram and the probability for this event is given by Lemma 1.

The algorithms proceed by aligning the pattern against the text and then shifting the pattern forward. Let us call an alignment bad if the last q -gram of the alignment matches the pattern in any position with at most k mismatches (for FFAST and Algorithm 1) or differences (for Algorithms 2, 3, and 4). All other alignments are good. The probability that an alignment is bad is at most $ma/(q\sigma^{dq}) < ma/\sigma^{dq}$.

In a good alignment all the algorithms will read the last q characters of the alignment and conclude that a shift of length $m - q + 1$ can be made. Thus the work done in a good alignment is bounded by $\mathcal{O}(q)$. If the alignment is bad and the last q -gram of the alignment matches the end of the pattern with at most k mismatches, FFAST

bad good **good** **good** **bad** **bad** **bad** good **good** **bad** **bad** **good** **good** ...

Fig. 6 Dividing the search phase into subphases when $q = 2$. The alignments, whose type influences the division, are shown in bold face

will compare the whole alignment against the pattern and Algorithm 1 will compare the rest of the pattern against the text and count the number of mismatches. Thus the work in a bad alignment can be bounded by $\mathcal{O}(m)$ in FFAST and Algorithm 1. In Algorithms 2, 3, and 4, the dynamic programming table will be built, if the last q -gram of the alignment matches the end of the pattern with less than k differences, so the work on bad alignments is bounded by $\mathcal{O}(m^2)$. The work in bad alignments can thus be expressed as $\mathcal{O}(m^B)$ where $B = 1$ for FFAST and Algorithm 1 and $B = 2$ for Algorithms 2, 3, and 4.

Let us then divide the search phase of the algorithms into subphases. Let a_i , $i = 1, 2, \dots$ be the alignments of the algorithms. The first subphase starts with a_1 . Let a_s be the first alignment of a subphase. The first good alignment in the sequence a_{s+qj} , $j = 0, 1, \dots$, ends this subphase. If a_e is the last alignment of a subphase, then a_{e+1} starts a new subphase. Thus each subphase consists of X groups of q alignments and one good alignment where $X \geq 0$ is a random variable. Each of the X groups of q alignments starts with a bad alignment and the rest $q - 1$ alignments may be of any type. Figure 6 shows an example of dividing the alignments into subphases.

The type of an alignment following a group of q alignments is independent of the first alignment of the group because the pattern has been shifted by at least q positions between them and the type of an alignment is determined solely by the last q -gram of the alignment. If $q \leq m - q + 1$, the type of an alignment after a good alignment is also independent of the good alignment, i.e. the q -gram determining the type of the next alignment contains only characters that have not been previously read. Because each subphase contains at least one good alignment, the text of length n will surely be covered after $\mathcal{O}(n/(m - q + 1))$ subphases.

The work done by the algorithms in one subphase of searching will be less than

$$\begin{aligned} & \mathcal{O}(q) \cdot P(X = 0) + \sum_{i=1}^{\infty} \left(\mathcal{O}(q) + i \cdot q \cdot \mathcal{O}(m^B) \right) \cdot P(X = i) \\ &= \mathcal{O}(q) + \sum_{i=1}^{\infty} i \cdot q \cdot \mathcal{O}(m^B) \cdot P(X = i) \\ &\leq \mathcal{O}(q) + q \cdot \mathcal{O}(m^B) \sum_{i=1}^{\infty} i \left(\frac{ma}{\sigma^{dq}} \right)^i. \end{aligned}$$

This sum will converge if $ma/\sigma^{dq} < 1$ or equally if $q > (1/d) \log_{\sigma}(ma)$ and then

$$\begin{aligned} \mathcal{O}(q) + q \cdot \mathcal{O}(m^B) \sum_{i=1}^{\infty} i \left(\frac{ma}{\sigma^{dq}} \right)^i &= \mathcal{O}(q) + q \cdot \mathcal{O}(m^B) \frac{\frac{ma}{\sigma^{dq}}}{(1 - ma/\sigma^{dq})^2} \\ &= \mathcal{O}(q) + q \cdot \mathcal{O}(m^B) \frac{ma \cdot \sigma^{dq}}{(\sigma^{dq} - ma)^2}. \end{aligned}$$

If we choose $q \geq \frac{C}{d} \log_{\sigma}(ma)$ where $C > 1$ is a constant then $\sigma^{dq} > ma$. Thus $\sigma^{dq} - ma = \Omega(\sigma^{dq})$ and therefore

$$\frac{1}{\sigma^{dq} - ma} = \mathcal{O}\left(\frac{1}{\sigma^{dq}}\right).$$

Now the work done by the algorithm in one subphase is less than

$$\begin{aligned} \mathcal{O}(q) + q \cdot \mathcal{O}(m^B) \frac{ma \cdot \sigma^{dq}}{(\sigma^{dq} - ma)^2} &= \mathcal{O}(q) \left(1 + \mathcal{O}\left(\frac{m^{B+1}a \cdot \sigma^{dq}}{\sigma^{2dq}}\right)\right) \\ &= \mathcal{O}\left(q \cdot \frac{m^{B+1}}{\sigma^{dq}}\right) = \mathcal{O}(q) \end{aligned}$$

if $C \geq B + 1$.

There are $\mathcal{O}(n/(m - q + 1))$ subphases and the average complexity of one subphase is $\mathcal{O}(q)$. Overall the average case complexity of the algorithms is thus

$$\mathcal{O}\left(\frac{n}{m - q + 1} \cdot q\right)$$

if $\frac{B+1}{d} \log_{\sigma}(ma) < q \leq m - q + 1$. The latter condition, $q \leq m - q + 1$, is equal to $q \leq (m + 1)/2$ and thus $m - q + 1 = \Omega(m)$ and the average case complexity becomes $\mathcal{O}(nq/m)$. Additionally $q \geq k/c$ so a safe choice for q is $q = \frac{B+1}{d} \log_{\sigma}(ma) + k/c = \Theta(\log_{\sigma} m + k)$ and then the average case complexity of the algorithms is $\mathcal{O}(n(\log_{\sigma} m + k)/m)$. An appropriate q exists if $k/c < (m + 1)/2$ and $(B + 1)/d \log_{\sigma}(ma) < (m + 1)/2$. The condition $k/c < (m + 1)/2$ becomes $k/(m + 1) < 1/2 - \mathcal{O}(1/\sqrt{\sigma})$ if we choose $c < 1 - e/\sqrt{\sigma}$ which also assures that $d > 0$. The latter condition $(B + 1)/d \log_{\sigma}(ma) < (m + 1)/2$ is equal to $\sigma^{m+1} > (ma)^{2(B+1)/d}$ which holds asymptotically for any σ and m . $\square$

The average complexity of the approximate string matching problem was proved to be $\Theta(n(\log_{\sigma} m + k)/m)$ by Chang and Marr [6] so our algorithms are average-optimal for an appropriate choice of q when $k/m < 1/2 - \mathcal{O}(1/\sqrt{\sigma})$.

Recall that the preprocessing time of our algorithms is $\mathcal{O}(m\sigma^q)$. With the choice $q = \Theta(k + \log_{\sigma} m)$, the preprocessing time becomes $\mathcal{O}(\sigma^{\Theta(k)} m^{\Theta(1)})$.

Algorithm 1 uses two preprocessed tables, M and S_q , both of which are of size σ^q . During the preprocessing phase we will additionally need space for the dynamic programming table which requires $\mathcal{O}(mq)$ space. Thus the total space requirement for Algorithm 1 is $\mathcal{O}(\sigma^q + mq) = \mathcal{O}(\sigma^{\Theta(k)} m^{\Theta(1)} + m \log_{\sigma} m + mk)$.

Algorithm 2 uses similar preprocessed tables and a similar dynamic programming table in the preprocessing phase as Algorithm 1 but additionally it needs to build the dynamic programming table in the searching phase requiring an extra $\mathcal{O}(m^2)$ space. Thus the space complexity of Algorithm 2 is $\mathcal{O}(\sigma^{\Theta(k)} m^{\Theta(1)} + m \log_{\sigma} m + mk + m^2)$.

Algorithm 3 adds another structure to those of Algorithm 2. It also stores the last row of the dynamic programming table for each q -gram. The additional space needed for this is $\mathcal{O}(m\sigma^q) = \mathcal{O}(\sigma^{\Theta(k)} m^{\Theta(1)})$. Thus the asymptotic space complexity of Algorithm 3 is also $\mathcal{O}(\sigma^{\Theta(k)} m^{\Theta(1)} + m \log_{\sigma} m + mk + m^2)$.

Algorithm 4 uses the same structures as Algorithm 2 except that it stores only the two last rows of the dynamic programming table during searching. Thus the asymptotic space complexity is $\mathcal{O}(\sigma^{\Theta(k)} m^{\Theta(1)} + m \log_{\sigma} m + mk)$.

6 Experimental Results

Tests were run on an AMD Athlon 1.0 GHz dual core CPU with 2 GB of memory, 64 kB L1 cache and 512 kB L2 cache. The computer was running Linux 2.6.23. The algorithms were written in C and compiled with the gcc compiler. For comparison in the k -mismatch case we used the following algorithms:

- ABM: The original ABM algorithm.
- FFAST: Our implementation of FFAST.
- FN: The mismatch version of the algorithm by Fredriksson and Navarro [8].

For the k -difference problem, we compared Algorithms 2 and 3 against the following algorithms:

- ABM: A version of ABM for the k -difference problem.
- Myers: Myers algorithm [12] is a linear time bit-parallel algorithm for patterns shorter than the computer word.
- BYP: The algorithm by Baeza-Yates and Perleberg [4] divides the pattern into smaller pieces so that if the pattern occurs at some position, at least one of the pieces must have an exact occurrence at that position. The algorithm then searches for exact matches of the pieces and verifies the occurrences found by the exact search.
- FN: The algorithm by Fredriksson and Navarro [8] for the k -difference problem.

All the results are shown with the q -value gaining the fastest searching speed in FFAST and our new algorithms if otherwise is not stated. The best q -value is generally the same for our algorithms and for FFAST. We tried several versions of the single pattern algorithm by Fredriksson and Navarro generally getting the best results with the version that reads the window backwards (–Sb option). Also for this algorithm we show the results with the best value for the parameter q . The other algorithms do not utilize the parameter q .

The searched text is a 22 MB long sequence of the fruit fly genome. The patterns have been extracted randomly from the text. Each pattern set consists of 200 different patterns of the same length, and they are searched sequentially.

Table 1 shows the search times for the original ABM, FFAST, FN, and Algorithm 1 in the k -mismatch problem and Fig. 7 further illustrates the results. The preprocessing times are shown in parenthesis. We used the code by Fredriksson and Navarro to measure the times for the FN algorithm. The code is designed for multiple patterns and the precision of measuring preprocessing time is not good enough to get reliable results for a single pattern. Thus the preprocessing times for the FN algorithm are not shown. Algorithm 1 is the fastest of the compared algorithms and it is generally about 30% faster than FFAST in the k -mismatch case for $k \in [1, 2]$. Also, the preprocessing phase of Algorithm 1 is 10 to 30 times faster than that of FFAST.

Table 1 Search times in seconds for k -mismatch, using best observed q -values for searching. The corresponding preprocessing times are shown in parenthesis. The run times of the algorithm by Fredriksson and Navarro shown here are for the options `-Sb`. The algorithm was slightly faster with options `-Sb -O` for $m = 10$, $k = 2$ yielding the runtime 73.83

m	ABM	FAAST		FN		Alg. 1	
	rt (s)	rt (s)	q	rt (s)	q	rt (s)	q
$k = 1$							
10	53.98 (0.03)	16.70 (0.15)	5	26.11	5	10.97 (0.02)	5
15	51.16 (0.01)	10.80 (1.39)	6	13.98	5	7.32 (0.02)	5
20	50.57 (0.06)	8.06 (2.01)	6	9.99	6	5.59 (0.14)	6
25	51.57 (0.04)	6.64 (2.26)	6	7.92	6	4.55 (0.16)	6
30	51.69 (0.10)	5.68 (3.27)	6	6.00	6	3.97 (0.12)	6
35	51.59 (0.17)	4.92 (4.03)	6	5.75	6	3.43 (0.20)	6
40	49.51 (0.21)	4.43 (4.75)	6	5.34	5	2.99 (0.35)	6
$k = 2$							
10	102.72 (0.02)	26.25 (0.62)	6	74.63	5	16.15 (0.13)	6
15	99.92 (0.02)	16.46 (5.88)	7	25.06	7	10.50 (0.17)	6
20	95.30 (0.05)	11.79 (8.16)	7	14.01	9	8.28 (0.14)	6
25	97.23 (0.06)	9.80 (10.74)	7	9.99	8	7.09 (0.18)	6
30	96.88 (0.13)	8.38 (13.41)	7	8.06	8	6.28 (0.22)	6
35	95.59 (0.09)	7.41 (16.00)	7	7.51	9	5.86 (0.16)	6
40	94.49 (0.24)	6.74 (18.66)	7	6.65	9	5.30 (1.21)	7

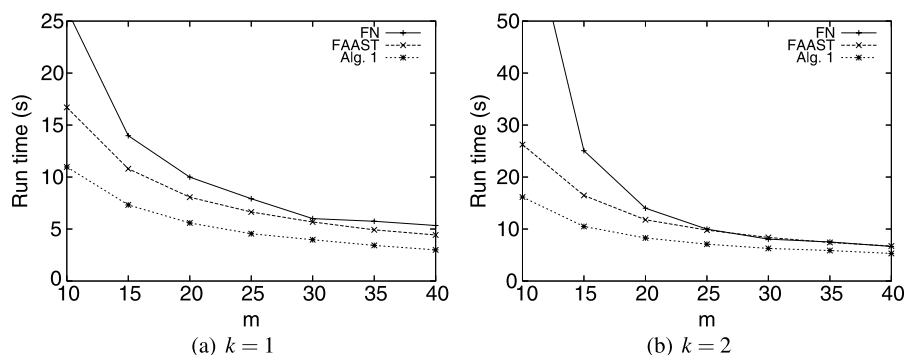


Fig. 7 Run time comparison of the algorithms for the mismatch problem

Experimental results for the k -difference problem are shown in Table 2, and Fig. 8 further illustrates the results. In the k -difference problem, our new algorithms are faster than the Myers, BYP, and ABM algorithms. They are also faster than the FN algorithm with short patterns but with longer patterns the FN algorithm is faster. The basic version of the Myers algorithm is limited by the 32-bit word size, and it cannot handle patterns where $m > 32$. The modifications in Algorithm 3 decrease search time by 20–30%, when compared to Algorithm 2.

Table 2 Search times in seconds for k -difference, using best observed q -values. The run times of the algorithm by Fredriksson and Navarro shown here are for the options $-Sb$. For $k = 1$ the algorithm was slightly faster with options $-Sb -L$ for $m = 10$ yielding the runtime 52.07. For $k = 2$ the algorithm was faster with options $-Sf$ for $m = 10$ and with options $-Sb -O$ for $m = 15$ yielding the run times 112.30 and 59.98 respectively

m	ABM	Myers	BYP	FN		Alg. 2		Alg. 3	
	rt (s)	rt (s)	rt (s)	rt (s)	q	rt (s)	q	rt (s)	q
$k = 1$									
10	112.78	57.29	33.53	56.41	4	22.11	7	18.04	6
15	76.97	57.47	26.45	18.81	6	16.43	7	13.98	7
20	68.00	57.67	25.79	10.89	6	14.15	7	11.92	7
25	67.97	59.10	25.89	8.10	6	13.79	7	11.17	7
30	67.70	57.76	25.86	6.34	6	13.35	7	10.56	7
35	67.85	—	25.89	5.99	6	13.79	7	10.67	7
40	67.12	—	25.37	4.92	6	14.91	7	10.84	7
$k = 2$									
10	392.37	60.30	159.78	174.42	8	79.51	8	56.41	8
15	268.95	58.15	61.93	69.75	6	57.50	9	42.65	8
20	197.90	58.38	41.43	18.58	9	45.55	9	38.85	9
25	151.94	57.65	37.54	13.92	9	41.02	9	43.76	9
30	124.78	57.76	37.10	8.84	9	38.56	9	31.42	9
35	100.43	—	38.23	8.27	9	36.89	9	29.47	9
40	86.95	—	36.85	7.62	9	37.48	9	29.07	9

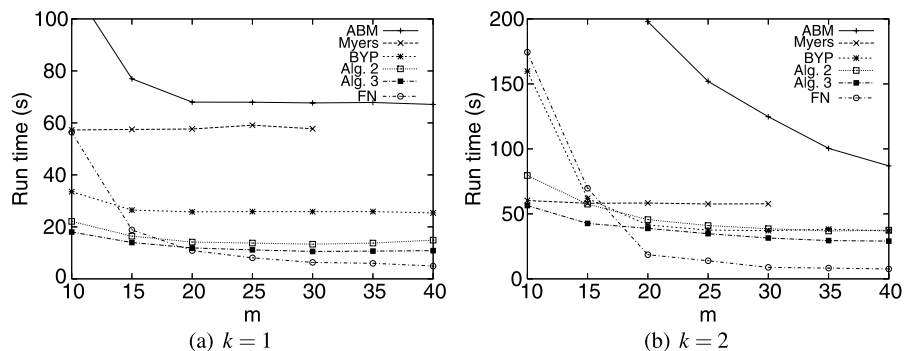


Fig. 8 Run time comparison of the algorithms for the difference problem

The effect of increasing the precalculated edit distance table size, and thus increasing preprocessing time with a large q -value is shown in Table 3. With small values of q , the search time decreases as the amount of preprocessing increases, but after a certain limit increasing the q -value will begin to slow down the search.

In the implementation of Algorithm 2, preprocessing is optimized by generating the q -grams in lexicographic order and recalculating the dynamic programming table

Table 3 Search times for k -difference, with different q -values ($k = 1$, $m = 20$)

q	Preprocessing		Search	
	Alg. 2	Alg. 3	Alg. 2	Alg. 3
2	< 0.01	0.01	4578.31	3500.39
3	0.01	0.02	1053.72	748.52
4	0.03	0.05	236.29	158.90
5	0.04	0.15	66.19	44.76
6	0.23	0.57	23.72	16.59
7	1.27	2.37	14.15	11.92
8	4.83	9.78	19.08	17.14
9	19.29	39.80	28.23	28.13
10	77.06	159.41	34.51	34.54

only for those characters that differ from the previous q -gram while Algorithm 3 needs to do this recursion twice, once to generate the normal dynamic programming table and once to calculate the reversed one. Thus the preprocessing times in Table 3 are longer for Algorithm 3 than for Algorithm 2.

7 Concluding Remarks

We have presented improved variations of the approximate Boyer-Moore algorithm for small alphabets for both the k -mismatch problem and the k -difference problem. We have also shown that the algorithms are average-optimal if we choose $q = \Theta(\log_{\sigma} m + k)$.

Acknowledgements We thank Janne Auvinen for implementing a part of the algorithms. We would also like to thank the anonymous reviewers for useful suggestions that helped to improve this paper.

References

1. Arlazarov, V., Dinic, E., Kronrod, M., Faradzev, I.: On economic construction of the transitive closure of a directed graph. Dokl. Acad. Nauk SSSR **194**, 487–488 (1970) (in Russian). English translation in Sov. Math. Dokl. **11**, 1209–1210 (1975)
2. Baeza-Yates, R., Gonnet, G.: A new approach to text searching. Commun. ACM **35**(10), 74–82 (1992)
3. Baeza-Yates, R., Gonnet, G.: Fast string matching with mismatches. Inf. Comput. **108**(2), 187–199 (1994)
4. Baeza-Yates, R., Perleberg, C.: Fast and practical approximate string matching. Inf. Process. Lett. **59**(1), 21–27 (1996)
5. Boyer, R., Moore, J.: A fast string searching algorithm. Commun. ACM **10**(20), 762–772 (1977)
6. Chang, W., Marr, T.: Approximate string matching and local similarity. In: Proceedings of the 5th Symposium on Combinatorial Pattern Matching. LNCS, vol. 807, pp. 259–173. Springer, Berlin (1994)
7. El-Mabrouk, N., Crochemore, M.: Boyer-Moore strategy to efficient approximate string matching. In: Proceedings of 7th Symposium on Combinatorial Pattern Matching. LNCS, vol. 1075, pp. 24–38. Springer, Berlin (1996)
8. Fredriksson, K., Navarro, G.: Average-optimal single and multiple approximate string matching. ACM J. Exp. Algorithm. **9**(4) (2004)

9. Horspool, N.: Practical fast searching in strings. *Softw. Pract. Experience* **10**, 501–506 (1980)
10. Liu, Z., Chen, X., Borneman, J., Jiang, T.: A fast algorithm for approximate string matching on gene sequences. In: *Proceedings of 16th Symposium on Combinatorial Pattern Matching. LNCS*, vol. 3537, pp. 79–90. Springer, Berlin (2005)
11. Masek, W., Paterson, M.: A faster algorithm for computing string edit distances. *J. Comput. Syst. Sci.* **20**, 18–31 (1980)
12. Myers, G.: A fast bit-vector algorithm for approximate string matching based on dynamic programming. *J. ACM* **46**(3), 395–415 (1999)
13. Navarro, G.: A guided tour to approximate string matching. *ACM Comput. Surv.* **33**(1), 31–88 (2001)
14. Navarro, G., Raffinot, M.: Fast and flexible string matching by combining bit-parallelism and suffix automata. *ACM J. Exp. Algorithm.* **5**(4) (2000)
15. Navarro, G., Sutinen, E., Tanninen, J., Tarhio, J.: Indexing text with approximate q -grams. In: *Proceedings of 11th Symposium on Combinatorial Pattern Matching. LNCS*, vol. 1848, pp. 350–363. Springer, Berlin (2000)
16. Rice, P., Longden, I., Bleasby, A.: EMBOS: The European molecular biology open software suite. *Trends Genet.* **16**(6), 276–277 (2000)
17. Tarhio, J., Ukkonen, E.: Approximate Boyer-Moore string matching. *SIAM J. Comput.* **22**, 243–260 (1993)
18. Wu, S., Manber, U., Myers, E.: A subquadratic algorithm for approximate limited expression matching. *Algorithmica* **15**(1), 50–67 (1996)